



SDEV 2451

Rapid Backend App Development

Global State and Authentication - 5

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- Nested Serializers
- ViewSet with Authentication
- Static to Dynamic Components
- Events API and React Query
- Route Parameters and Protected Routes
- Event Detail Components
- CartContext — Client-Side State
- Checkout Summary
- Key Takeaways

Nested Serializers

Nested serializers compose related objects into a single API response so the frontend never needs extra requests.

```
# articles/serializers.py

class AuthorSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ("id", "username")

class TagSerializer(serializers.ModelSerializer):
    class Meta:
        model = Tag
        fields = ("id", "name")

class ArticleSerializer(serializers.ModelSerializer):
    author = AuthorSerializer(read_only=True)
    tags = TagSerializer(many=True, read_only=True)

    class Meta:
        model = Article
        fields = ("id", "title", "author", "tags")
```

Nested Serializers (continued)

- Declaring nested fields explicitly overrides DRF's default behaviour of returning only foreign key IDs
- `read_only=True` marks the field as output-only — attempts to write through it raise a validation error
- `many=True` tells DRF the field is a one-to-many relationship and serializes the queryset into a list
- Small, focused serializers (`AuthorSerializer` , `TagSerializer`) can be reused in other serializers without duplication

ViewSet with Authentication

`ModelViewSet` combined with `DefaultRouter` generates all standard REST routes; `IsAuthenticated` secures them.

```
# articles/views.py
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.select_related("author").prefetch_related("tags")
    permission_classes = (IsAuthenticated,)

    def get_serializer_class(self):
        if self.action == "list":
            return ArticleListSerializer
        return ArticleDetailSerializer

# articles/urls.py
router = DefaultRouter()
router.register("articles", ArticleViewSet, basename="article")
urlpatterns = router.urls
```

ViewSet with Authentication (continued)

- `DefaultRouter` generates `GET /articles/` (list) and `GET /articles/<pk>/` (detail) among other routes automatically
- `select_related("author")` fetches the author in the same SQL JOIN — without it, serializing 10 articles fires 10 extra queries
- `prefetch_related("tags")` fetches all tags in a single follow-up query and maps them in Python — used for M2M/reverse FK
- `IsAuthenticated` requires a valid `Authorization: Bearer` header; DRF returns `401 Unauthorized` before any view logic runs
- `get_serializer_class` is the standard DRF pattern for choosing a different serializer per action

Static to Dynamic Components

Build with hardcoded data first to validate layout, then replace static values with props and real API data.

```
// Step 1 - static placeholder
function BookCard() {
  return (
    <div className="card bg-base-100 shadow">
      <div className="card-body">
        <h2 className="card-title">
          The Great Gatsby
        </h2>
        <p className="text-sm text-base-content/60">
          F. Scott Fitzgerald • 1925
        </p>
        <button className="btn btn-primary btn-sm">
          View
        </button>
      </div>
    </div>
  )
}
```

```
// Step 2 - accept a prop
function BookCard({ book }) {
  return (
    <div className="card bg-base-100 shadow">
      <div className="card-body">
        <h2 className="card-title">
          {book.title}
        </h2>
        <p className="text-sm text-base-content/60">
          {book.author} • {book.year}
        </p>
        <Link
          to={`/${book.id}`}
          className="btn btn-primary btn-sm"
        >
          View
        </Link>
      </div>
    </div>
  )
}
```



Static to Dynamic Components (continued)

- Hardcoding data first lets you validate the grid layout and card design before any API work begins
- The repeated static `<BookCard />` is intentional — it will be replaced by a `.map()` over real data in the next step
- Using `Link` from React Router renders an `<a>` tag that navigates without a full page reload, preserving React Query's cache
- `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` makes the grid responsive across mobile, tablet, and desktop widths

Events API and React Query

A thin API function throws on failure so React Query can catch it; a custom hook encapsulates `useQuery` for clean imports.

```
// src/api/books.js
import apiClient from './client'

export async function fetchBooks() {
  const res = await apiClient('/books/')
  if (!res.ok) {
    throw new Error('Failed to fetch books.')
  }
  return res.json()
}

export async function fetchBook(id) {
  const res = await apiClient(`/books/${id}/`)
  if (!res.ok) {
    throw new Error('Failed to fetch book.')
  }
}
```

```
// src/hooks/useBooks.js
import { useQuery } from '@tanstack/react-query'
import { fetchBooks } from '../api/books'

export function useBooks() {
  return useQuery({
    queryKey: ['books'],
    queryFn: fetchBooks,
  })
}
```

```
// src/hooks/useBook.js
import { fetchBook } from '../api/books'

export function useBook(id) {
  return useQuery({
    queryKey: ['books', id],
    queryFn: () => fetchBook(id),
  })
}
```



Events API and React Query (continued)

- Throwing on `!res.ok` converts a non-OK HTTP response into a JS error — React Query catches it and sets `isError: true`
- `queryKey: [ 'books' ]` is the cache key — two components calling `useBooks()` share one network request, no duplicates
- Including the ID in the key (`[ 'books', id]`) gives each detail page its own cache entry, preventing stale cross-page data
- Encapsulating `useQuery` in a custom hook means components only import from `../hooks/` — if the query key changes, only the hook file needs updating

Route Parameters and Protected Routes

`useParams` reads dynamic URL segments; `ProtectedRoute` redirects unauthenticated users before the page mounts.

```
// src/pages/BookDetailPage.jsx
import { useParams } from 'react-router-dom'

function BookDetailPage() {
  const { id } = useParams() // '/books/3' → { id: "3" }
  return <p>Book ID: {id}</p>
}

// src/App.jsx – protecting routes
<Route
  path="/"
  element={<ProtectedRoute><BooksListPage /></ProtectedRoute>}
/>
<Route
  path="/books/:id"
  element={<ProtectedRoute><BookDetailPage /></ProtectedRoute>}
/>
```

Route Parameters and Protected Routes (continued)

- `useParams()` returns an object whose keys match named route segments — `/books/:id` gives `{ id: "3" }`
- `ProtectedRoute` reads `user` from `AuthContext`; if `null` it renders `<Navigate to="/login" replace />` before the page mounts
- `replace` swaps the current history entry so the user cannot press Back to reach the protected page again without logging in
- Wrapping at the route level in `App.jsx` keeps protection in one place — one-line change to add or remove auth from any page
- A frontend route guard protects the UI; `IsAuthenticated` on the viewset protects the data — you need both

Event Detail Components

Compose a detail page from focused sub-components, each starting with hardcoded data before accepting props.

```
// src/components/ArticleDetailsHeader.jsx
function ArticleDetailsHeader({ article }) {
  return (
    <div className="bg-base-100 shadow rounded-box p-6 mb-6">
      <h1 className="text-3xl font-bold mb-2">{article.title}</h1>
      <p className="text-base-content/60 mb-4">{article.publishedAt}</p>
      <div>
        <p className="font-semibold">{article.author.name}</p>
        <p className="text-sm text-base-content/60">{article.author.bio}</p>
      </div>
    </div>
  )
}
```

Event Detail Components (continued)

- Start with hardcoded values so layout and visual hierarchy are validated before any data-fetching is introduced
- `rounded-box` applies the DaisyUI theme's standard border radius, keeping card corners consistent across the UI
- Each component has one job: `ArticleDetailsHeader` renders the header, `TagList` arranges tags — composing them keeps each file small
- Replace static constants with data from `useBook(id)` — `isLoading` and `isError` guards prevent the `.map()` from running on `undefined`

CartContext — Client-Side State

`CartContext` stores in-progress selections as a keyed object; `CartProvider` sits outside `BrowserRouter` so state persists across navigation.

```
// src/contexts/CartContext.jsx
export function CartProvider({ children }) {
  const [items, updateCartItems] = useState({})

  function updateCart({ bookId, tierId, tierName, price, quantity }) {
    const key = `${bookId}-${tierId}`
    updateCartItems((prev) => {
      if (quantity === 0) {
        const { [key]: _, ...rest } = prev
        return rest
      }
      return { ...prev, [key]: { bookId, tierId, tierName, price, quantity } }
    })
  }

  return <CartContext.Provider value={{ items, updateCart }}>{children}</CartContext.Provider>
}
```


CartContext — Client-Side State (continued)

- `items` is a plain object keyed by ``${bookId}-${tierId}`` — guarantees uniqueness even when two books share the same tier name
- The functional updater `(prev) ⇒ ...` reads the latest state value, avoiding stale closure bugs when the user clicks rapidly
- `const { [key]: _, ...rest } = prev` removes a zero-quantity entry using destructuring — the cart stays clean with no zero-qty lingering
- Place `CartProvider` inside `AuthProvider` but outside `BrowserRouter` — navigating away does not unmount the provider or reset the cart
- `useCart` lives in `hooks/` alongside `useAuth` — components always import hooks from `../hooks/`, never from `../contexts/`

Checkout Summary

Convert the cart object to an array with `Object.values`, map over it for display, and derive the grand total with `reduce`.

```
// src/pages/CheckoutPage.jsx
function CheckoutPage() {
  const { items } = useCart()
  const cartItems = Object.values(items)
  const grandTotal = cartItems.reduce(
    (sum, item) => sum + item.quantity * parseFloat(item.price),
    0
  )

  return (
    <div>
      {cartItems.length === 0 ? (
        <p>Your cart is empty.</p>
      ) : (
        cartItems.map((item) => (
          <div key={`_${item.bookId}-${item.tierId}`}>
            <p>{item.tierName} × {item.quantity}</p>
            <p>${(item.quantity * parseFloat(item.price)).toFixed(2)}</p>
          </div>
        ))
      )}
    </div>
  )
}
```



Checkout Summary (continued)

- `Object.values(items)` converts the cart object to an array for mapping — keys were only needed for $O(1)$ lookups inside the context
- `parseFloat(item.price)` converts DRF's decimal string (e.g. `"150.00"`) to a number for arithmetic; `.toFixed(2)` formats it back for display
- `reduce` starting at `0` accumulates the grand total in one pass — an empty cart correctly produces `$0.00` without a separate guard
- Conditionally render a "Go to Checkout" Link when `Object.keys(items).length > 0` — no extra state, just a derived boolean

Key Takeaways

- Lock both layers independently — a `ProtectedRoute` guards the UI while `IsAuthenticated` on the viewset guards the data; one without the other leaves a gap
- Separate server state from client state — React Query owns data that lives on the server (articles, article details); Context owns data that lives in the browser session (auth, cart)
- Build static first, then connect — every component in this example started with hardcoded data, letting the layout and hierarchy be validated before any API work began
- Keep components focused — `TicketTierSelector` selects a quantity, `TicketTierList` arranges selectors, the page fetches data and composes the view; each has one job
- The cart context, API client, and auth flow built here carry forward unchanged into the next example for order submission



Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS